INPUT / RESOLUTION / BOUNDARY / EVIDENCE

Path Traversal & File Inclusion Explained

Follow the name, prove the file operation, and protect the boundary

FILE LOOKUP

Which object is selected?

FILE AUTHORITY

Which operation is allowed?

Follow the name from input to the actual file operation.

Find the weakness. Prove the effect. Verify the repair.

Practice with real files and harmless owned fixtures.

Haroon Shahid

First edition / Version 1.0 / 1 November 2026

Before you begin

Path Traversal & File Inclusion Explained

Follow the name, prove the file operation, and protect the boundary

By Haroon Shahid. First edition, version 1.0. 1 November 2026. Prepared with AI assistance.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under LICENSE.txt. The separately identified lab code is MIT licensed. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Pathwright uses actual small file operations in a run-owned temporary tree: controlled reads, writes, unlink, ZIP staging, link and descriptor comparisons where supported, fixed original Python fragment execution and manual jobs. Actor switching, global state, name swaps, revocation and fault injection are privileged fixtures. PHP, Windows filesystem, framework, proxy, cloud, container and agent integrations are review guidance, not deployed projects in this kit. TAR is parsed but not extracted. The model is not a production filesystem security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format. On narrow screens, swipe sideways within wide diagrams and tables. No full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package includes all 56 chapters, answered reviews, the offline reading edition, practice workbooks, reference sheets, and the local Pathwright lab.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. Understand files, folders and names 7
02. Follow a web request to a file operation 9
03. Locate the starting directory and the root 11
Sources and further reading 13

Introduction

A guide opens. An archive is extracted. A page includes a view. These ordinary features all depend on selecting the right resource with the right authority.

The difficult questions come next. Does the name stay inside its permitted directory? Does it still refer to the same object when opened? Are the selected bytes displayed as data or interpreted as code?

This book starts with files and requests. You need no prior penetration-testing experience. Learn the terms when they become useful, predict a result, inspect a controlled example and explain the decision in your own words.

One connected service

Cairn Archive is fictional. Mira uses Blue; Ben uses Amber. Public guides, private dummy notes, original fragments and generated uploads give us clear objects and rules. Pathwright, the optional lab, performs real small file operations inside a temporary tree owned by that run.

A known harmless marker is enough to prove an important boundary when the rule and control are clear. You do not need a real secret or a dramatic payload.

Four questions to carry

1. Input: what can this actor influence, and in which representation?

2. Resolution: which transformations and lookup rules select the object?

3. Boundary: which location, permission, resource and consumer rules must hold?

4. Evidence: which observed result and permitted control establish the claim?

These questions connect parent-directory examples to links, mutable names, archives, code loaders and workers. The exact input can change. The reasoning stays useful.

Use the learning kit

Each chapter has an analogy, two teaching/revision diagrams, a takeaway and three questions. Explained answers are in Appendix A. Companion workbooks add predictions, missions, worked review cases, evidence templates and a mastery tracker.

The catalog links primary standards, manuals, framework guidance and historical advisories. Consulted means targeted reading, not an audit of every page or deployment of every stack. The research cutoff is 1 November 2026.

Keep the evidence honest

Validation names the executed environment and limits. Python fragment execution is not PHP. Windows string parsing is not NTFS execution. A privileged link or swap fixture does not prove attacker control elsewhere. Real integrations need their actual deployment tests.

Use only systems and identities you own or are authorized to assess. Start with a valid control. Follow the name. Finish with a result you can defend.

Learning roadmap

Choose a starting route

- New learner: Chapters 1-7, then the first missions. Explain files, paths, requests, actors and proof.

- Developer: foundations, then Chapters 22-35 and 43-48. Write selection, lookup and publication contracts.

- Authorized tester: Chapters 8-21, then lifecycle sections. Produce paired tests and bounded file evidence.

- Architect: Chapters 35-49 after the evidence model. Map workers, identities, mounts and consumers.

Learning dependencies / file authority from input to operation



Read left to right within each stage, then follow the connector to the next row. Each later stage uses the earlier mental model.

Eight stages, one connected model

Stage	Chapters	Demonstrate before moving on
Files and evidence	1-7	Explain path versus object, read versus include and permitted controls
Finding and proof	8-14	Prove a local parent, absolute or transformation failure with a bounded claim
Actual resolution	15-21	Explain platform syntax, links, snapshots, handles and mutation prerequisites
File lifecycle	22-28	Trace read, write, delete, extraction, resource limits and publication
Inclusion	29-35	Identify the code consumer and every additional chain prerequisite
Architecture	36-42	Bind authority across exports, caches, proxies, cloud keys and tools
Safer operations	43-49	Choose supported controls and verify the actual integration
Independent review	50-56	Review a new design, report accurately and maintain the boundary

Practice order

Begin with the guide and object comparisons. Add path representations, then links and the controlled name swap. Move to writes and deletion only inside the resettable fixture tree. Study extraction before the batch/fault experiments. Complete the worker comparisons and independent capstone last.

The prediction workbook is for a first attempt before revealing an answer. The review clinic is synthetic architecture practice, not additional executed deployments. The guided missions name the exact model operations and explain capability requirements.

Retain the model

At the end of a stage, redraw its main flow from memory. Explain one valid operation, one failure and one limitation. Revisit after a day and about a week, then apply the concept to a different authorized example. Change the spacing when your recall suggests it.

Finish with evidence

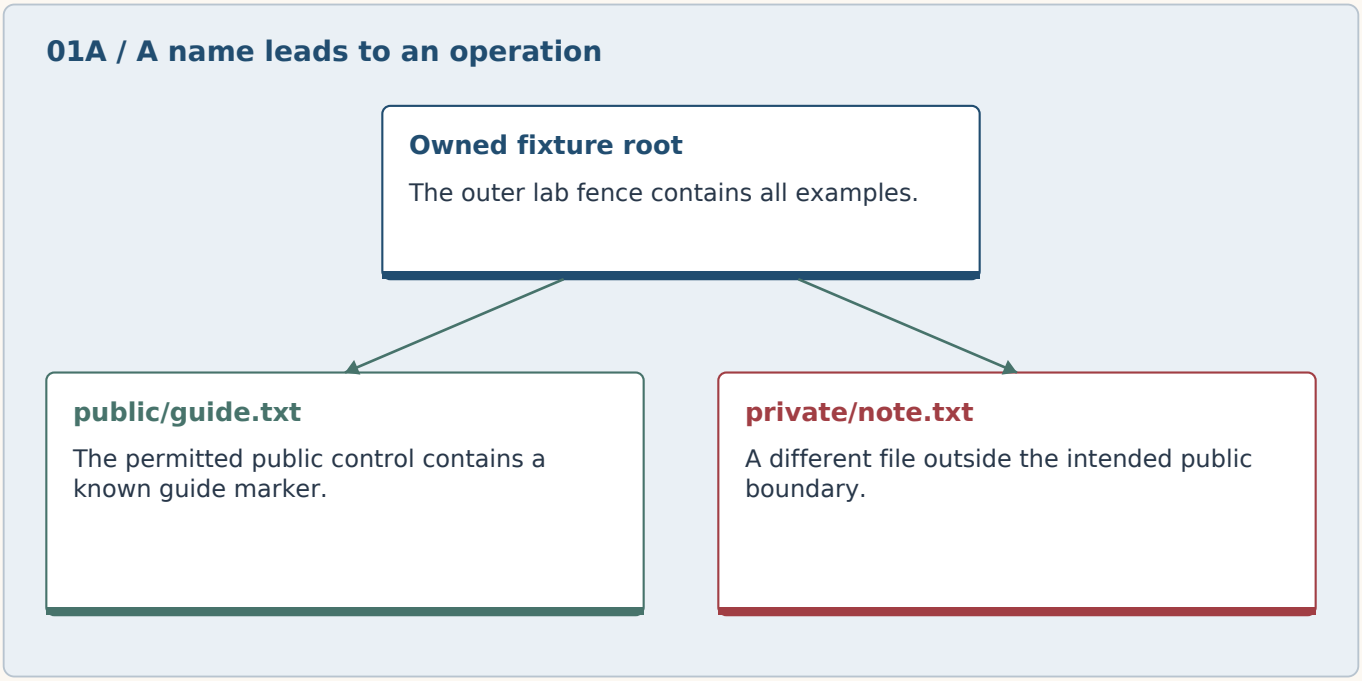
Use the mastery tracker to collect an explanation, a controlled proof, a repair contract and an independent review. Completing a checklist is not certification. The evidence should show what you can reason through without looking at the worked answer.

I. UNDERSTAND FILES, PATHS AND EVIDENCE

01. Understand files, folders and names

Learn to: Explain stored bytes, directory entries and a path before testing any input.

Think of it this way: A library label tells a clerk where to look. It is not the book itself.



Separate the name, the object and the operation before deciding whether a boundary failed.

Start with stored information

A **file** holds bytes: small units used to represent text, pictures, programs and other information. A text editor shows some bytes as letters. A filename extension such as .txt is a naming convention. It does not prove what the bytes mean or which program will process them.

A **directory**, often called a folder, organizes names. A **path** is a sequence that helps a program find an object through those names. The name and object are different. A file can be renamed while its contents stay unchanged. Later chapters show how more than one name can reach the same object.

Our fictional service, Cairn Archive, offers public guides and private invoices. Its practice tree contains public/guide.txt and private/note.txt. The private note is a harmless marker, not a host secret. The rule is simple: an anonymous-style guide selector must not become permission to read the private note.

The important questions

When a web feature handles a name, ask what it selects and what the server will do with that selection. A preview reads bytes. An upload creates or replaces an object. A cleanup feature removes names. An include operation may ask an interpreter to process a selected file as program material. These are different operations.

Do not start by memorizing ../. First explain the intended directory, permitted objects and operation. A name containing unusual characters is only input. The vulnerability appears when the application uses it to cross a required boundary.

Try the first observation

In Pathwright, read guide.txt with the direct path policy. The response contains DUMMY-PUBLIC-GUIDE, its byte count and a hash. Those details come from a real small file. A successful response alone would not establish that fact. Keep this normal result as your first control.

01B / Input, resolution, boundary, evidence

Input	Resolution	Boundary	Evidence
A filename is a selector, not a permission.	The operating system resolves names to objects.	The allowed directory and operation form the rule.	A known dummy marker establishes a normal read.

Key takeaway. Separate the name, the object and the operation before deciding whether a boundary failed.

Check your understanding

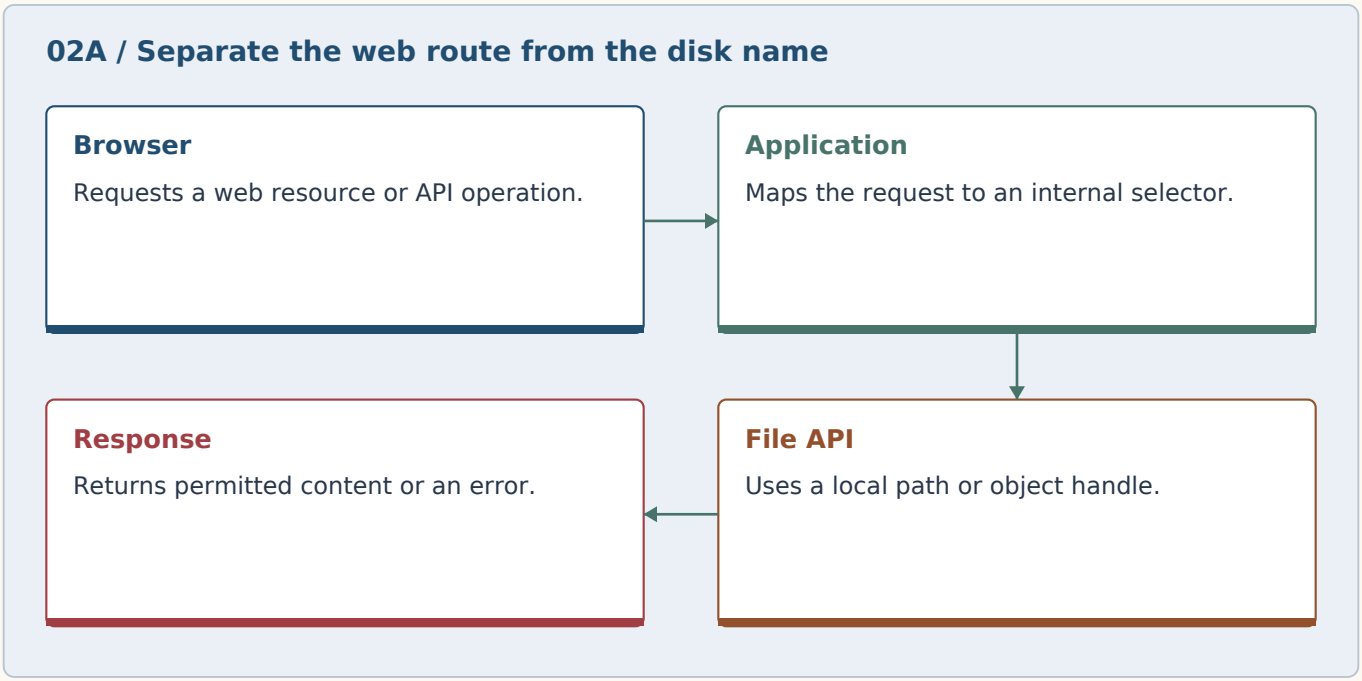
1.1 Does .txt prove that a file is harmless?	**1.2** Is a path the file itself?	**1.3** Why keep a normal guide read?

Answer explanations are in the full ebook. Sources: PATHRES, CWE22

02. Follow a web request to a file operation

Learn to: Distinguish a browser URL, application route and server filesystem path.

Think of it this way: A street address for a shop is different from the shelf number used inside it.



Trace the request into the actual accepting file operation instead of treating a URL as a disk map.

Two different address spaces

A **client** asks for a service. A **server** receives the request and decides what work to perform. A browser is one kind of client. HTTP carries request methods, targets, headers and sometimes a body. The response carries a status, headers and usually content. HTTPS protects the connection; it does not decide which server file a user may select.

A URL such as https://archive.example/download?id=guide identifies a web endpoint. The application might map guide to a database object, a local file or a cloud object. There is no general rule that /download is a real directory on disk. The browser does not automatically know the server's folder layout.

Find the conversion

The security-relevant step often looks like this:

```
request field -> application selection -> file API -> response
```

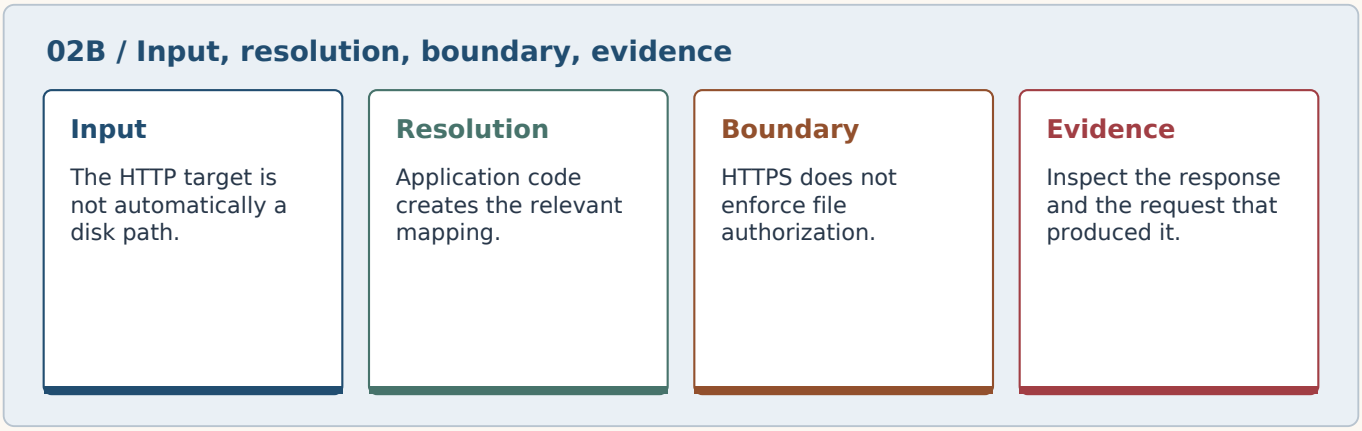
A **file API** is the interface a program uses to read, create or otherwise access files. The application may build a path before calling that interface. This is where a user-controlled name can gain more meaning than intended.

In Pathwright, the reading field travels in a JSON request body. The server receives the literal string and explicitly chooses a reading policy. This makes the decoding stages visible. It does not recreate every web server's URL handling.

Inspect rather than assume

Use your browser’s Network panel or an authorized proxy to identify the actual request. Record the method, field, actor and response. Change one field and preserve a valid control. A visible page may make additional API calls; the request that returns the file could differ from the first navigation.

A 200 response might be a login page or a generic error document. Inspect content, media type and the known marker. A filename echoed in JSON is not proof that the server opened it.



Key takeaway. Trace the request into the actual accepting file operation instead of treating a URL as a disk map.

Check your understanding

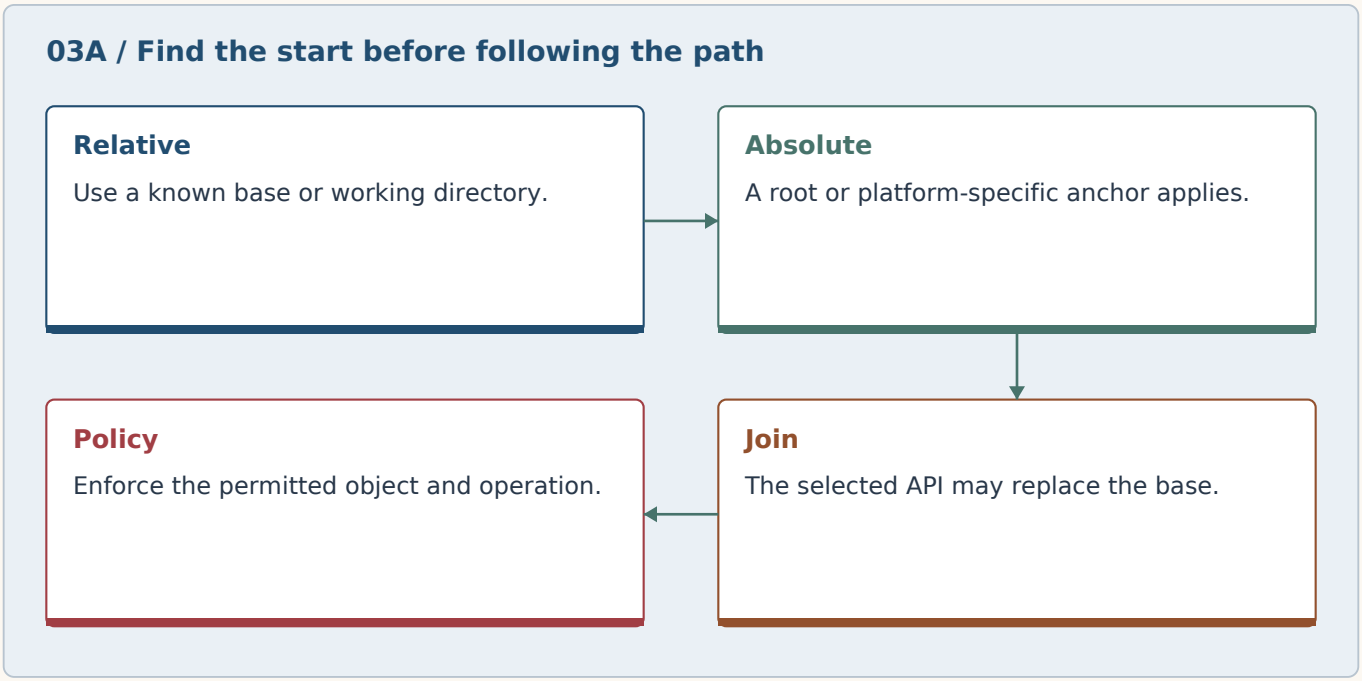
2.1 Does HTTPS prevent traversal?	**2.2** Must /download exist on disk?	**2.3** Does status 200 prove a file read?

Answer explanations are in the full ebook. Sources: HTTP, URL, WSTG

03. Locate the starting directory and the root

Learn to: Predict relative and absolute lookup, including current working directory effects.

Think of it this way: Turn left means nothing until you know where the journey starts.



A path needs a starting point; joining a base does not prove that lookup stays beneath it.

Relative names need a starting point

A **relative path** is interpreted from a starting directory. On a POSIX-style system, guide.txt might mean a file in the process's current working directory. An application can instead join it to a chosen directory such as /srv/archive/public.

The **current working directory** is process state. It need not be the source code directory. A service manager, test runner or deployment script can start the same program elsewhere. An assumption that worked on a developer's laptop can therefore select a different file in deployment.

An **absolute path** supplies a root. On POSIX systems it begins with /. The root is the process's visible filesystem root, which may differ inside a container or other namespace. Windows has additional drive and share forms, covered later.

Joining is not confinement

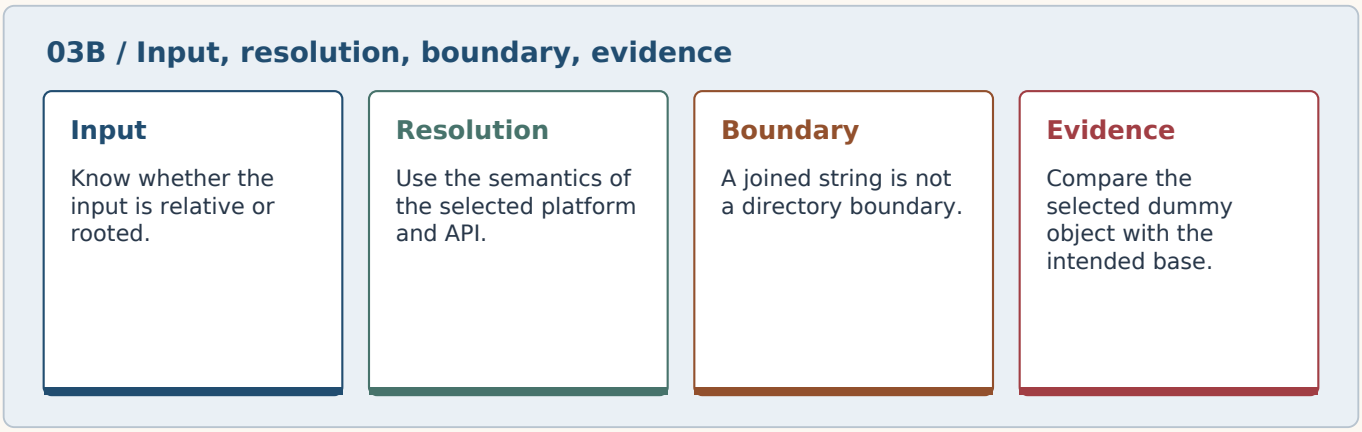
Some APIs discard the earlier base when a later argument is absolute. In Python, joining a Path base with an absolute Path selects the absolute argument. Other languages have their own join and resolve rules. Do not transfer one language's behavior to every API.

Pathwright exposes its temporary fixture root in privileged state. The owned absolute-path button selects that root's private/note.txt. The weak direct-join reader reaches it. The outer lab fence still blocks files outside the run-owned tree. That fence is separate from the intentionally broken public-directory rule.

Design a clear contract

If an application needs a relative name, reject rooted, drive-relative and ambiguous forms according to the actual target platform. Then enforce the directory and object policy when the operation occurs. Rejecting an absolute spelling alone does not stop parent traversal, symlinks or unauthorized objects inside the directory.

A safer product may accept an object ID instead. The server chooses the physical location. That removes a large naming problem, while leaving permission checks essential.



Key takeaway. A path needs a starting point; joining a base does not prove that lookup stays beneath it.

Check your understanding

3.1 Is the working directory always the script directory?	**3.2** Does every join API treat absolute arguments alike?	**3.3** What does the lab absolute-path example prove?

Answer explanations are in the full ebook. Sources: PATHRES, OSPATH, NODEPATH

Sources and further reading

Research cutoff: 1 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

PATHRES - Linux path_resolution(7)

Consulted targeted documentation: Starting points, component lookup and process roots.

https://man7.org/linux/man-pages/man7/path_resolution.7.html

CWE22 - MITRE CWE-22

Consulted targeted documentation: Pathname restriction weakness and effects.

https://cwe.mitre.org/data/definitions/22.html

HTTP - IETF RFC 9110: HTTP Semantics

Consulted targeted documentation: Methods, response semantics and representation handling.

https://www.rfc-editor.org/rfc/rfc9110

URL - WHATWG URL Living Standard

Consulted targeted documentation: URL parsing and path interpretation; not an OS path specification.

https://url.spec.whatwg.org/

WSTG - OWASP WSTG v4.2: Directory Traversal / File Include

Consulted targeted documentation: Versioned test guidance; historical examples require current runtime verification.

https://wstg.owasp.org/v4.2/4-Web_Application_Security_Testing/05-Authorization_Testing/01-Testing_Directory_Traversal_File_Include/

OSPATH - Python 3.12 os.path

Consulted targeted documentation: Join, normalizing and component versus textual prefix behavior.

https://docs.python.org/3.12/library/os.path.html

NODEPATH - Node.js Path API

Consulted targeted documentation: Native and explicit Windows/POSIX path semantics; live documentation changes.

https://nodejs.org/api/path.html